

Technical Documentation

Real-Time Event Streaming with NiFi, Kafka, and HDFS

Pipeline Architecture & Implementation Details

Objective: Design and implement a robust, decoupled data pipeline capable of ingesting batch CSV files, transforming them into individual JSON events, streaming them through a Kafka broker, and dynamically partitioning the aggregated results into a Hadoop Distributed File System (HDFS).

1 Pipeline Explanation

The architecture is designed as a decoupled, event-driven ingestion engine. It utilizes an intermediary Message Broker (Apache Kafka) to separate the data extraction and transformation phase from the final storage phase. This ensures that the system can handle sudden spikes in e-commerce traffic without overwhelming the final Hadoop Data Lake.

1.1 Core Components

- **Data Source (Python Simulator):** A script generating randomized, semi-structured e-commerce transactions in CSV format. It intentionally injects malformed data (e.g., NaN prices, missing IDs, corrupted strings) to simulate real-world data quality issues.
- **Ingestion & Transformation (NiFi Producer):** Monitors a local directory, chunks large incoming CSV files, enforces schema validation to drop bad records, converts the valid CSV rows into JSON objects, and pushes them to Kafka.
- **Message Broker (Kafka Cluster):** A highly available distributed log consisting of 3 partitions and 3 replicas under the topic `ecommerce_json_events`.
- **Batching & Storage (NiFi Consumer):** Subscribes to the Kafka topic, aggregates the high-velocity JSON stream into optimized 128MB blocks to prevent the “Small File Problem”, and writes them to HDFS.

2 Data Flow & Process Groups

To maintain operational clarity, the NiFi canvas is divided into distinct Producer and Consumer flows.

2.1 Overall Pipeline View

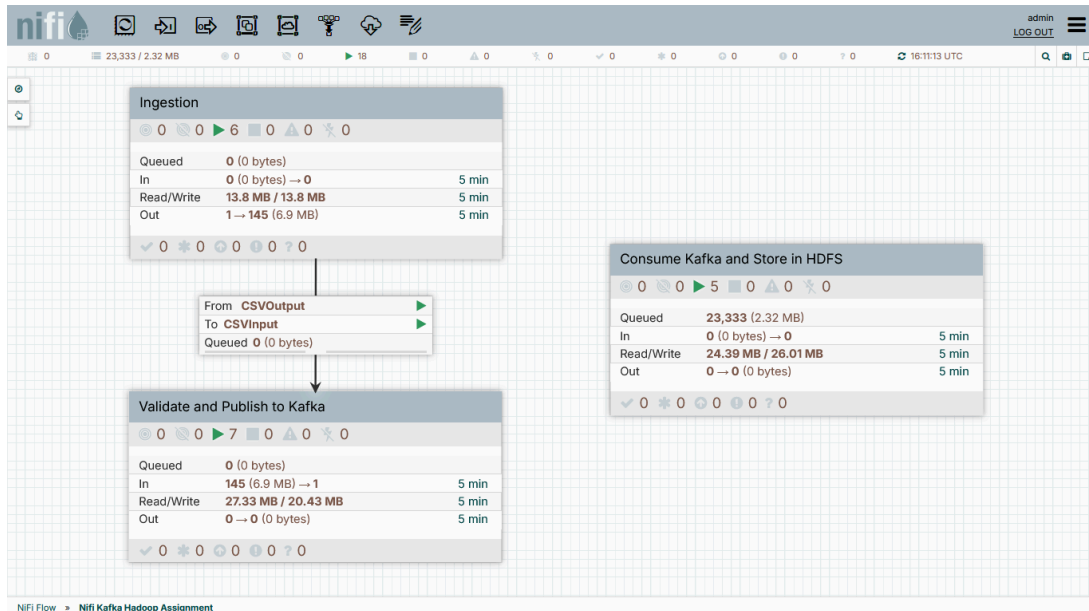


Figure 1: The root NiFi canvas displaying the decoupled Process Groups.

2.2 Phase 1: File Ingestion & Chunking

The pipeline begins by capturing the generated CSV files using the GetFile processor. Because the generated files can vary in size, a SplitText processor immediately breaks them down into uniform 64KB fragments. This prevents memory bottlenecks downstream when converting formats.

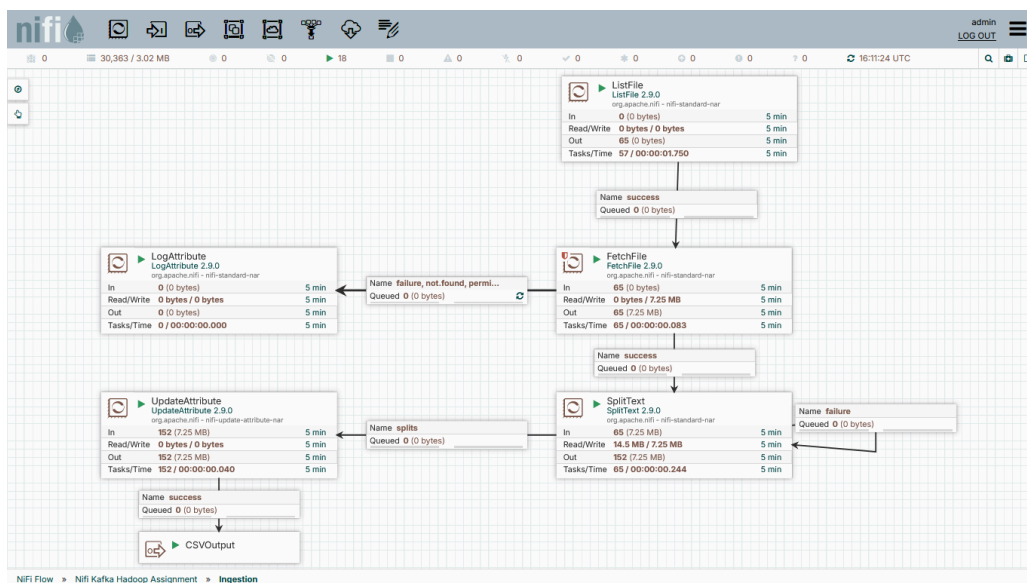


Figure 2: File Ingestion and Chunking via GetFile and SplitText.

2.3 Phase 2: Schema Validation & Transformation

Data quality is enforced using NiFi's Record-Oriented processors.

- **ValidateRecord** acts as the gatekeeper. It reads the CSV chunks against a predefined Avro schema. Any row that contains invalid data types (such as the injected "CORRUPTED_DATA_STRING" or "NaN" instead of a float) is instantly routed to a Dead Letter Queue.
- **ConvertRecord** takes the successfully validated records and translates them from CSV format into individual JSON objects, making them ready for real-time event streaming.

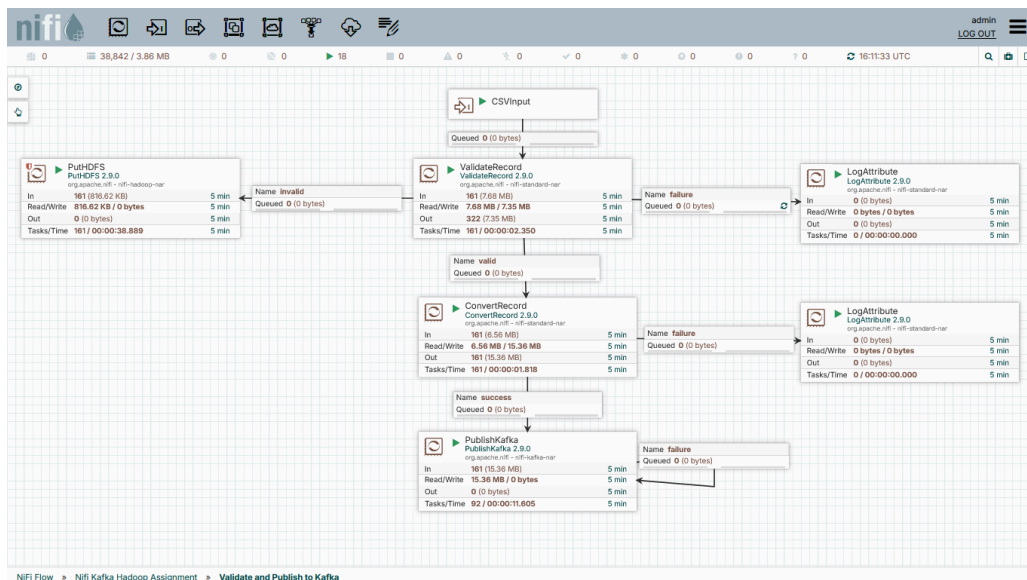


Figure 3: Data Validation, Transformation, and Kafka Publishing flow.

2.4 Phase 3: Event Streaming (Kafka)

The PublishKafka processor takes the validated JSON records and pushes them into the Kafka broker.

- **Delivery Guarantee:** Set to all to ensure the broker acknowledges the message across all 3 replicas before NiFi considers the transaction complete, guaranteeing zero data loss.
- **Format:** Records are published as single-line JSON objects, decoupled from the original CSV batch context.

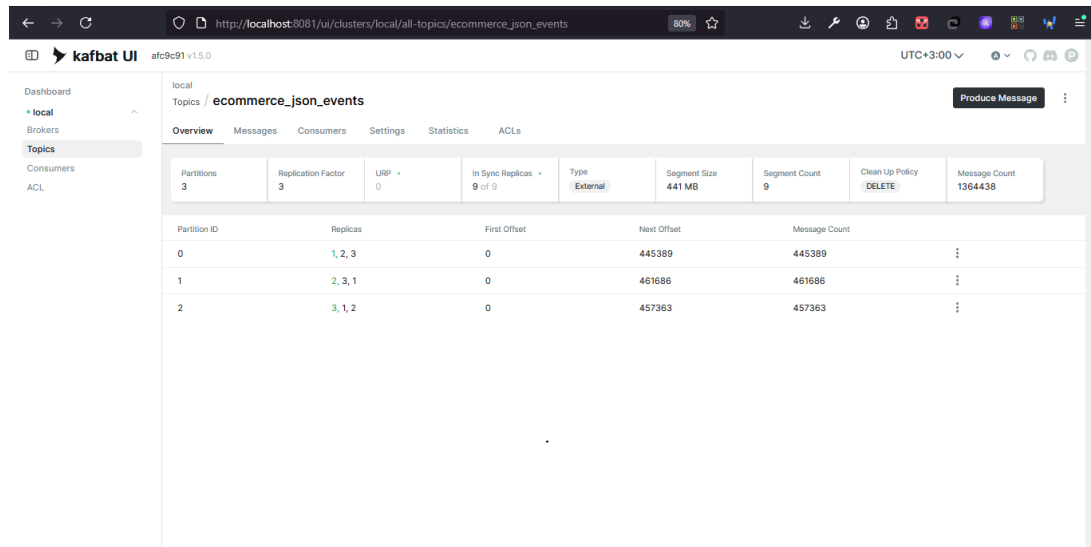


Figure 4: Kafka Topic overview showing 3 Partitions and 3 Replicas.

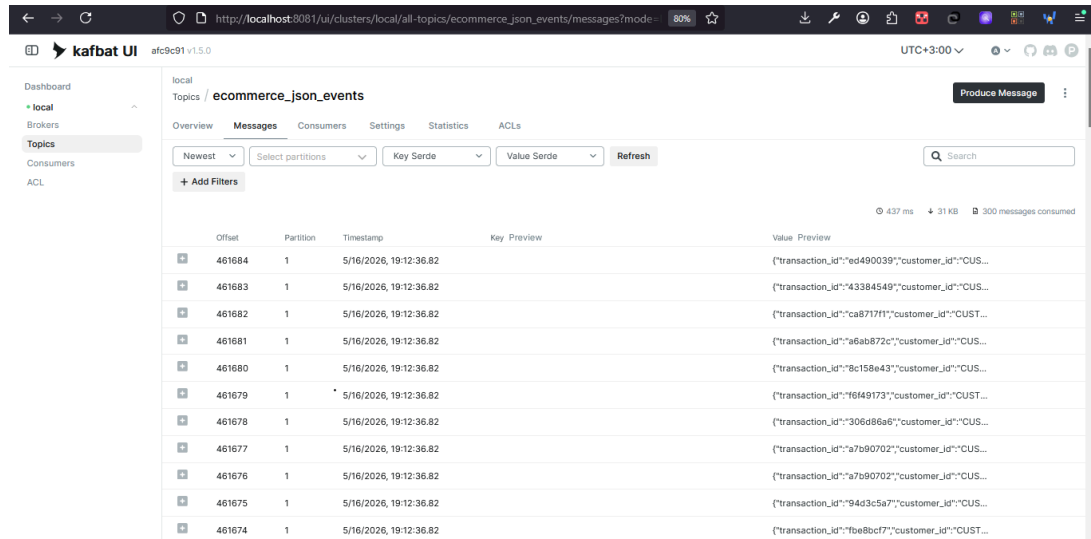


Figure 5: Real-time JSON events successfully streaming into the Kafka topic.

2.5 Phase 4: Batching & HDFS Storage

A decoupled ConsumeKafka processor constantly polls the broker.

- **Micro-Batching:** Instead of writing thousands of tiny JSON files to Hadoop, the MergeContent processor buffers the real-time stream until it reaches 128MB.
- **Dynamic Routing:** The PutHDFS processor leverages NiFi Expression Language (`${now()}`) to dynamically generate the output directory paths, allowing for fully automated data partitioning.

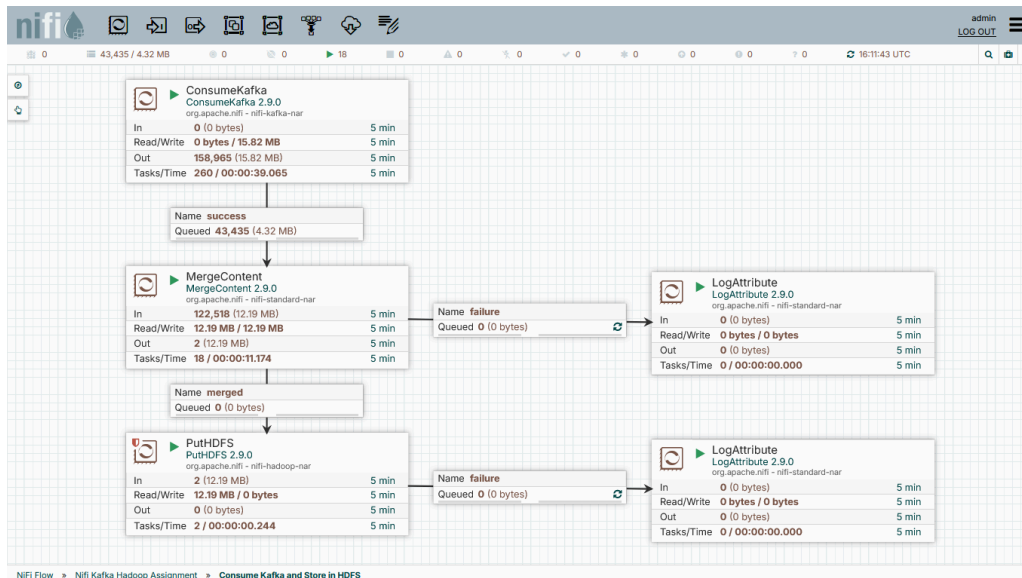


Figure 6: Kafka Consumption, Micro-batching, and HDFS Storage flow.

3 Architectural Strategies

3.1 Error Handling Approach (Dead Letter Queue)

Silent data loss is prevented at every stage.

- **Validation Failures:** Records that fail schema validation in `ValidateRecord` are explicitly routed to an invalid relationship, acting as a Dead Letter Queue (DLQ). These records can be logged, alerted on, or stored in a separate directory for manual review.
- **Network Failures:** If `PublishKafka` or `PutHDFS` encounter network timeouts, they utilize NiFi's default "Penalize FlowFile" strategy. The data is temporarily paused in the queue while the processor yields, preventing infinite aggressive retries that could crash the system.

3.2 Partitioning Strategy

Partitioning is implemented at two distinct levels to optimize both streaming and storage:

- **Kafka Partitioning:** The `ecommerce_json_events` topic is created with 3 partitions. Since no specific message key is provided, the `DefaultPartitioner` utilizes a Round-Robin strategy to distribute the load evenly across all available broker nodes.
- **HDFS Time-Based Partitioning:** The `PutHDFS` processor is configured with the dynamic directory property `/data/year=${now():format('yyyy')}/month=${now():format('MM')}/day=${now():format('dd')}/`. This natively creates "Hive-style partitions", meaning downstream analytical engines like Trino, Spark, or Hive can query the data optimally by date without needing an active metastore during ingestion.

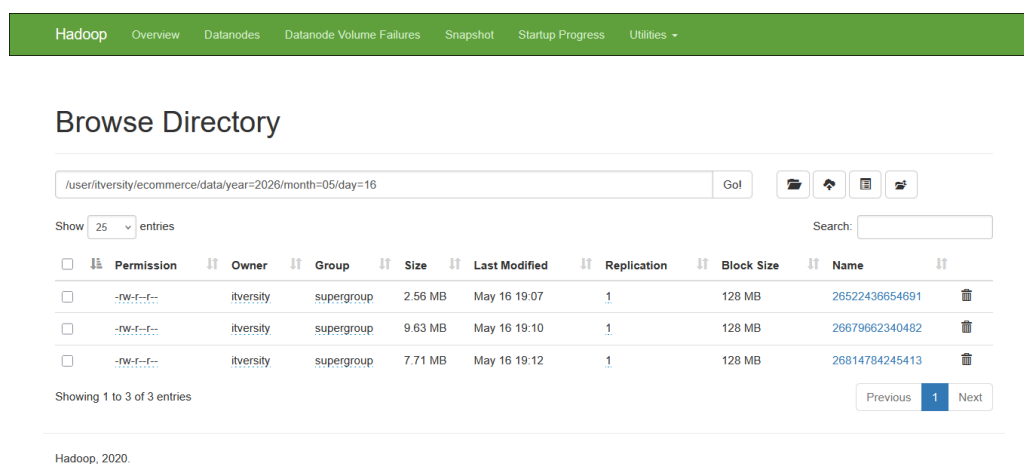


Figure 7: Dynamic Time-Based Partitioning natively created in HDFS.

3.3 Challenges Faced During Implementation

- **Docker Networking & DNS:** Initial attempts to connect NiFi to HDFS resulted in `java.net.ConnectException: Connection refused`. This occurred because the Hadoop `core-site.xml` natively bound the NameNode to `localhost:9000`, making it unreachable from the NiFi Docker container.

- **Resolution:** The issue was resolved by leveraging Docker's internal DNS network. The NameNode configuration was updated via sed to bind to 0.0.0.0:9000 (allowing external traffic), and the NiFi PutHDFS processor was updated to target hdfs://itvdelab:9000.

4 Sample Data Transformations

To verify data integrity, samples were captured at the ingestion source (Python generator) and the final destination (HDFS Data Lake).

4.1 Raw Simulated Data (Before Transformation)

This sample displays the messy CSV data dropped by the Python generator, containing injected malformed records.

```
Generated ecommerce_batch_20260516_161031_52.csv with 3135 records.
Generated ecommerce_batch_20260516_161033_53.csv with 1576 records.
Generated ecommerce_batch_20260516_161037_54.csv with 2537 records.
Generated ecommerce_batch_20260516_161041_55.csv with 3314 records.
Generated ecommerce_batch_20260516_161043_56.csv with 1750 records.
Generated ecommerce_batch_20260516_161048_57.csv with 1978 records.
Generated ecommerce_batch_20260516_161053_58.csv with 2962 records.
Generated ecommerce_batch_20260516_161055_59.csv with 2394 records.
Generated ecommerce_batch_20260516_161058_60.csv with 2596 records.
Generated ecommerce_batch_20260516_161103_61.csv with 3383 records.
Generated ecommerce_batch_20260516_161107_62.csv with 1612 records.
Generated ecommerce_batch_20260516_161111_63.csv with 3765 records.
Generated ecommerce_batch_20260516_161114_64.csv with 2867 records.
Generated ecommerce_batch_20260516_161118_65.csv with 1641 records.
Generated ecommerce_batch_20260516_161121_66.csv with 2927 records.
Generated ecommerce_batch_20260516_161126_67.csv with 3404 records.
Generated ecommerce_batch_20260516_161129_68.csv with 3659 records.
Generated ecommerce_batch_20260516_161132_69.csv with 3416 records.
Generated ecommerce_batch_20260516_161137_70.csv with 1992 records.
Generated ecommerce_batch_20260516_161140_71.csv with 3844 records.
Generated ecommerce_batch_20260516_161144_72.csv with 3490 records.
Generated ecommerce_batch_20260516_161148_73.csv with 2344 records.
Generated ecommerce_batch_20260516_161151_74.csv with 1506 records.
Generated ecommerce_batch_20260516_161154_75.csv with 2819 records.
Generated ecommerce_batch_20260516_161157_76.csv with 1803 records.
Generated ecommerce_batch_20260516_161201_77.csv with 1947 records.
Generated ecommerce_batch_20260516_161205_78.csv with 3483 records.
Generated ecommerce_batch_20260516_161210_79.csv with 1623 records.
Generated ecommerce_batch_20260516_161215_80.csv with 2520 records.
Generated ecommerce_batch_20260516_161219_81.csv with 3636 records.
Generated ecommerce_batch_20260516_161221_82.csv with 3234 records.
Generated ecommerce_batch_20260516_161223_83.csv with 2069 records.
Generated ecommerce_batch_20260516_161226_84.csv with 1957 records.
Generated ecommerce_batch_20260516_161230_85.csv with 3416 records.
^C
Streaming generator stopped.
nifi@cbd1bbfd5bc7:/opt/nifi/nifi-current$
```

Figure 8: Raw generated CSV data prior to ingestion.

```
transaction_id,customer_id,amount,timestamp
c14a938b,CUST784,850.45,2026-05-16 20:15:30
d29b124a,CUST102,12.99,2026-05-16T20:15:33.124Z
e58f001c,CUST441,105.00,20260516201540,CORRUPTED_DATA_STRING
f88b9a11,INVALID,NaN,16/05/2026 20:15
```


4.2 Processed HDFS Data (After Transformation)

The final 128MB (I did lower the minimum size to 12MB for demo purposes) merged block in HDFS successfully dropped the corrupted and NaN rows via schema validation, cleanly mapping the remaining valid rows to JSON format.

```

itversity@itvdelab:~$ hdfs dfs -ls ecommerce/data/year=2026/month=05/day=16
Found 3 items
-rw-r--r-- 1 itversity supergroup 2685260 2026-05-16 16:07 ecommerce/data/year=2026/month=05/day=16/26522436654691
-rw-r--r-- 1 itversity supergroup 18099558 2026-05-16 16:10 ecommerce/data/year=2026/month=05/day=16/26679662340482
-rw-r--r-- 1 itversity supergroup 8083517 2026-05-16 16:12 ecommerce/data/year=2026/month=05/day=16/26814784245413
itversity@itvdelab:~$ hdfs dfs -get ecommerce/data/year=2026/month=05/day=16/26522436654691 ./day_16.json
itversity@itvdelab:~$ head day_16.json
{"transaction_id":"df86690d","customer_id":"CUST393","amount":1391.65,"timestamp":"20260516154349"}
{"transaction_id":"7954980e","customer_id":"CUST752","amount":113.78,"timestamp":"2026-05-16T15:43:49.387670Z"}
{"transaction_id":"ebd7d377","customer_id":"CUST697","amount":1470.42,"timestamp":"2026-05-16 15:43:49"}
{"transaction_id":"5300b83d","customer_id":"CUST931","amount":1272.15,"timestamp":"2026-05-16T15:43:49.387751Z"}
{"transaction_id":"b9de5769","customer_id":"CUST539","amount":-999.99,"timestamp":"16/05/2026 15:43"}
{"transaction_id":"b5cd61bf","customer_id":"CUST718","amount":211.18,"timestamp":"20260516154349"}
{"transaction_id":"b5cd61bf","customer_id":"CUST718","amount":211.18,"timestamp":"20260516154349"}
{"transaction_id":"7625b109","customer_id":"CUST914","amount":751.39,"timestamp":"2026-05-16T15:43:49.387827Z"}
{"transaction_id":"1736ffd5","customer_id":"CUST858","amount":239.71,"timestamp":"20260516154349"}
itversity@itvdelab:~$

```

Figure 9: Final batched JSON data stored in HDFS, cleanly formatted and validated.

JSON

```

{"transaction_id": "c14a938b", "customer_id": "CUST784", "amount": 850.45, "timestamp": "2026-05-16 20:15:30"}
{"transaction_id": "d29b124a", "customer_id": "CUST102", "amount": 12.99, "timestamp": "2026-05-16T20:15:33.124Z"}

```

— End of Documentation —